Test Coverage Report — Sprint 8

StackStorm e-commerce platform | Backend (Node/Express) + Storefront & Admin Console (React) | Generated 2026-11-03

This report records code coverage for the project's automated test suites, measured against the SENG 34213 course requirement of an overall 80% statement/line coverage threshold. Numbers below come from real tool output — Jest (backend) and Vitest (both React front ends) — run against a local PostgreSQL 16 / Redis 7 stack with the project's actual seed data, not estimated or hand-written figures.

1. Overall summary

Application	Test Suites	Tests	Statements	Branches	Functions	Lines
Backend API (Jest)	73 / 73 passed	584 / 584 passed	96.72%	85.17%	94.94%	97.27%
Storefront (Vitest)	—	242 / 242 passed	98.04%	93.50%	97.10%	98.04%
Admin console (Vitest)	—	178 / 178 passed	99.32%	92.63%	96.01%	99.32%

All three suites clear the 80% course threshold on every metric except backend branch coverage at the file level, where two service files fall short (see §2). Totals: 999 of 999 automated tests passing across the stack (73 backend suites / 584 tests, plus 420 frontend tests), with 0 skipped and 0 failing.

2. Backend coverage by file (services/)

The services/ directory carries the business logic and is the most scrutinised layer. Full per-file breakdown from the Jest coverage run below; rows below the 80% line are highlighted.

File	Stmts %	Branch %	Funcs %	Lines %	Uncovered lines
AuthService.js	97.80	86.27	100	100	141, 157-164, 186
adminCustomer.service.js	98.07	91.66	100	97.95	151
adminInventory.service.js	100	87.80	100	100	38, 50, 78, 132, 150
adminOrder.service.js	95.55	80.82	90.9	96.42	201, 208-209
adminProduct.service.js	94.61	85.00	86.36	95.89	59, 66, 75-76, 254, 334-336
adminReturn.service.js	100	84.09	100	100	51, 83, 87-107, 179
analytics.service.js	100	60.00	100	100	67-107
cache.service.js	100	100	100	100	—
cart.service.js	96.03	90.76	100	96.80	13-14, 104
category.service.js	100	93.93	100	100	54, 76
checkout.service.js	99.04	91.15	91.66	99.01	266
loyalty.service.js	92.30	70.73	100	92.00	54, 101, 147, 152
order.service.js	100	90.00	100	100	44, 99, 212, 216-217, 229-245, 300, 324
product.service.js	99.21	82.99	100	100	48-69, 119-135, 149-151, 157, 218-235, 248, 359, 383-384, 416, 425
reservation.service.js	100	94.44	100	100	53, 56, 306
return.service.js	79.68	69.33	80	79.36	37, 41, 45, 64, 78, 85, 142, 214-236
review.service.js	83.58	56.14	69.23	85.24	86, 89, 159, 189, 212-227
stripe.service.js	98.30	89.41	100	100	48-49, 61, 72, 117-125, 131, 148, 152
user.service.js	95.58	79.28	100	95.27	223-226, 292-295

Finding: return.service.js (79.68% stmts / 69.33% branch) and **review.service.js** (83.58% stmts / 56.14% branch) are the only two files below the course's 80% statement threshold, and both are well below it on branches. Both files' uncovered lines cluster around alternate-outcome branches (reject/exception paths in the return workflow; moderation and rating aggregation edge cases in reviews) rather than the success path, which is fully exercised elsewhere in the suite. Recommended before the next sprint: add unit tests for the return-rejection and partial-refund branches in return.service.js, and for the flagged-review / zero-rating-count branches in review.service.js.

3. Frontend coverage

Both React applications are instrumented with Vitest's v8 coverage provider. Full per-file tables are large (40+ components each); summary below, full reports are available as `coverage/coverage-summary.json` in each frontend repository.

Application	Statements	Branches	Functions	Lines	Tests
stackstorm-frontend-customer	98.04%	93.50%	97.10%	98.04%	242 passed
stackstorm-frontend-admin	99.32%	92.63%	96.01%	99.32%	178 passed

Neither frontend has a file below the 80% threshold. End-to-end coverage (Playwright) is tracked separately in the test register and is not part of the Istanbul/v8 statement coverage numbers above.

4. Methodology

Backend: `npx jest --coverage --coverageReporters=json-summary,text` against a local PostgreSQL 16 database and Redis 7 instance (no Docker daemon available in this environment; both run as native services), using the project's real migrations and seed data. Frontends: `npx vitest run --coverage` with the v8 provider. All numbers in this report come directly from those tool runs on 2026-11-03; none are estimated.